Chủ đề 3: Xử lý sự kiện

Nội dung chính

3.1. Sử dụng các thành phần View

3.2. Resource trong ứng dụng Android

Hiểu cách sử dụng các thành phần View như TextView, Button, EditText, CheckBox, RadioButton, Spinner và AutoCompleteTextView để tạo giao diện người dùng tương tác trên Android.

Biết cách quản lý tài nguyên trong ứng dụng Android thông qua các tệp tài nguyên như chuỗi ký tự, màu sắc, hình ảnh, và bố cục, từ đó dễ dàng tùy chỉnh và hỗ trợ đa nền tảng.

Vận dụng các thành phần View và tài nguyên vào trong các ứng dụng thực tế, bao gồm việc xây dựng giao diện, xử lý sự kiện và tối ưu hóa trải nghiệm người dung.

Mục tiêu bài học

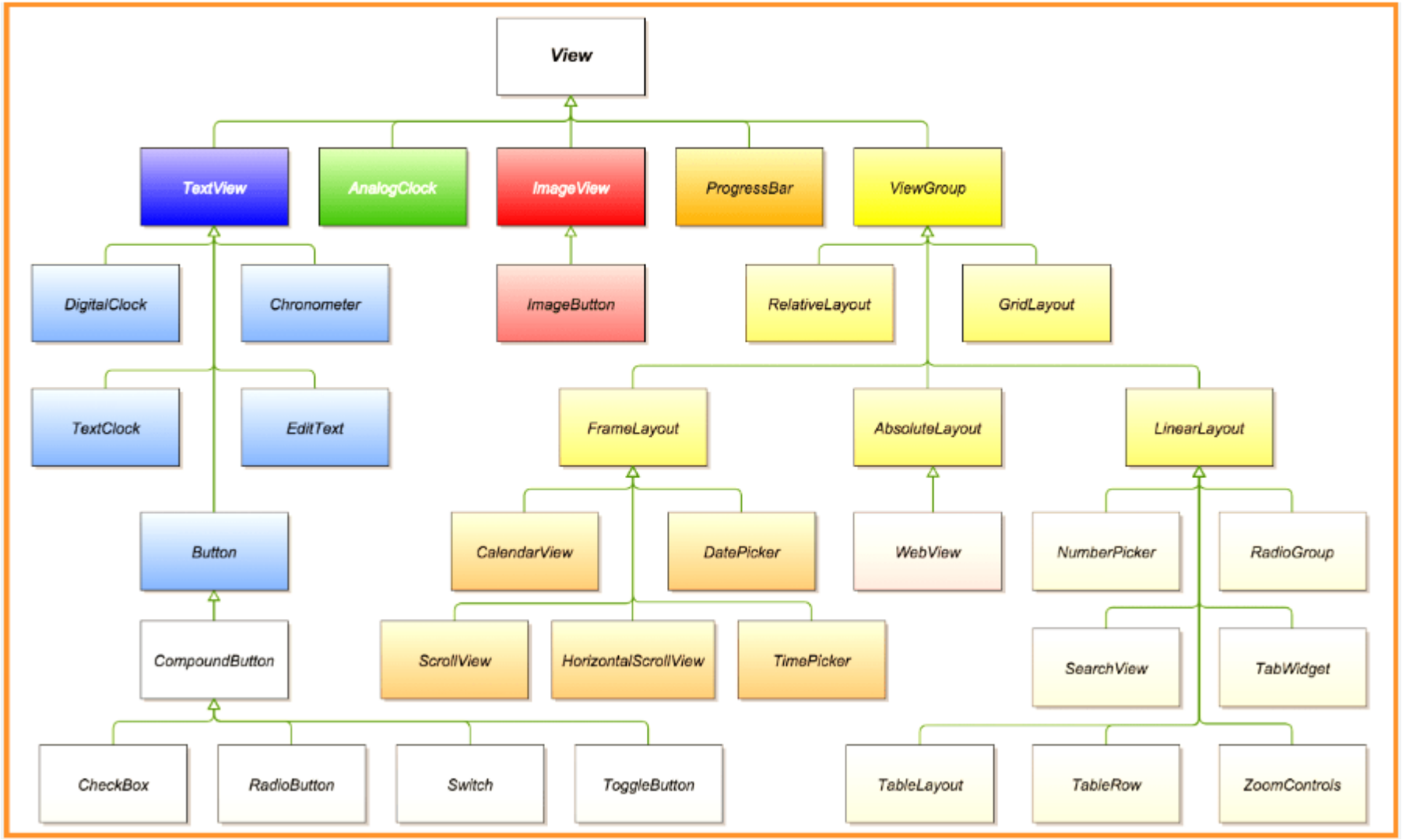
Hiểu cách sử dụng các thành phần View như **TextView, Button, EditText, CheckBox, RadioButton, Spinner, AutoCompleteTextView**.

Biết cách **quản lý tài nguyên** trong ứng dụng Android.

Vận dụng các thành phần View vào xây dựng **giao diện tương tác**.

Xử lý sự kiện và tối ưu hóa trải nghiệm người dùng.

Giới thiệu về thành phần View



TextView – Thành phần hiển thị văn bản

Dùng để hiển thị nội dung tĩnh hoặc động. Các thuộc tính quan trọng:

- android:text: Nội dung văn bản hiển thị.
- android:textSize: Kích thước chữ.
- android:textColor: Màu chữ.
- android:textStyle: Kiểu chữ (đậm, nghiêng).

```
<TextView
    android:id="@+id/txtExample"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Chào mừng đến với Android!"
    android:textSize="18sp"
    android:textColor="#FF0000"
    android:textStyle="bold" />
```

Button – Thành phần xử lý sự kiện nhấn

Kích hoạt hành động khi người dùng nhấn vào.Các thuộc tính quan trọng:

- android:text: Nhãn hiển thị trên Button.
- android:background: Đổi màu nền.
- android:onClick: Gọi hàm xử lý khi nhấn.

```
<Button
    android:id="@+id/btnExample"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Nhấn vào đây"
    android:background="@color/teal_200"
    android:onClick="onButtonClick" />
```

EditText – Thành phần nhập dữ liệu

Cho phép người dùng nhập văn bản.Các thuộc tính quan trọng:

- android:hint: Hiển thị gợi ý nhập liệu.
- android:inputType: Xác định loại dữ liệu nhập vào (text, number, password).
- android:maxLength: Giới hạn số ký tự.

```
<EditText
    android:id="@+id/edtInput"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Nhập nội dung..."
    android:inputType="textPassword"
    android:maxLength="12" />
```

CheckBox – Thành phần chọn nhiều giá trị

Cho phép chọn một hoặc nhiều giá trị.Các thuộc tính quan trọng:

- android:text: Nhãn của CheckBox.
- android:checked: Mặc định chọn hoặc không.

```
<CheckBox
    android:id="@+id/checkExample"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Chọn tôi"
    android:checked="false" />
```

RadioButton – Thành phần chọn một giá trị

Chỉ cho phép chọn một giá trị trong nhóm.Các thuộc tính quan trọng:

- android:text: Nhãn hiển thị.
- android:checked: Mặc định chọn hoặc không.

```
<RadioGroup
    android:id="@+id/radioGroup"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content">
    <RadioButton
        android:id="@+id/radio1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Lựa chọn 1"
        android:checked="true" />
</RadioGroup>
```

Giới thiệu về Resource trong Android

Resource (tài nguyên) là các dữ liệu không phải mã nguồn được sử dụng trong ứng dụng.Giúp dễ dàng thay đổi giao diện, ngôn ngữ mà không chỉnh sửa mã nguồn.

Các loại resource chính:

- Chuỗi (String): Văn bản hiển thị trong ứng dụng.
- Màu sắc (Color): Định nghĩa màu sử dụng trong UI.
- Hình ảnh (Drawable): Biểu tượng, ảnh nền.
- Giao diện (Layout): Cấu trúc UI của ứng dụng.

Resource Strings – Quản lý văn bản

Lưu trữ chuỗi văn bản trong tập tin res/values/strings.xml:

```
<resources>
    <string name="app_name">Ứng dụng Android</string>
    <string name="welcome_message">Chào mừng đến với ứng dụng!</string>
</resources>
```

Cách sử dụng trong mã nguồn:

```
txtResult.text = getString(R.string.welcome_message)
```

Giúp dễ dàng dịch ngôn ngữ và quản lý nội dung.

Resource Color – Định nghĩa màu sắc

Màu sắc được lưu trong res/values/colors.xml:

```
<resources>
    <color name="primary_color">#6200EE</color>
    <color name="secondary_color">#03DAC5</color>
</resources>
```

Sử dụng màu trong XML:

```
<TextView
    android:text="Chào mừng"
    android:textColor="@color/primary_color" />
```

Sử dụng màu trong Kotlin:

```
val color = ContextCompat.getColor(this, R.color.primary_color)
textView.setTextColor(color)
```

Resource Drawable – Quản lý hình ảnh

Hình ảnh và icon được lưu trong res/drawable/.

Các loại drawable phổ biến:

- Bitmap: PNG, JPG.
- Vector: SVG.
- Shape: Định nghĩa hình dạng (bo góc, viền, nền...).

Ví dụ sử dụng hình ảnh:

```
<ImageView
    android:src="@drawable/logo"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content" />
```

Resource Layout – Cấu trúc giao diện

Layout là file XML xác định bố cục UI trong res/layout/.
Ví dụ activity_main.xml:

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <TextView
        android:text="Chào mừng"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
</LinearLayout>
```

Quản lý Resource hiệu quả

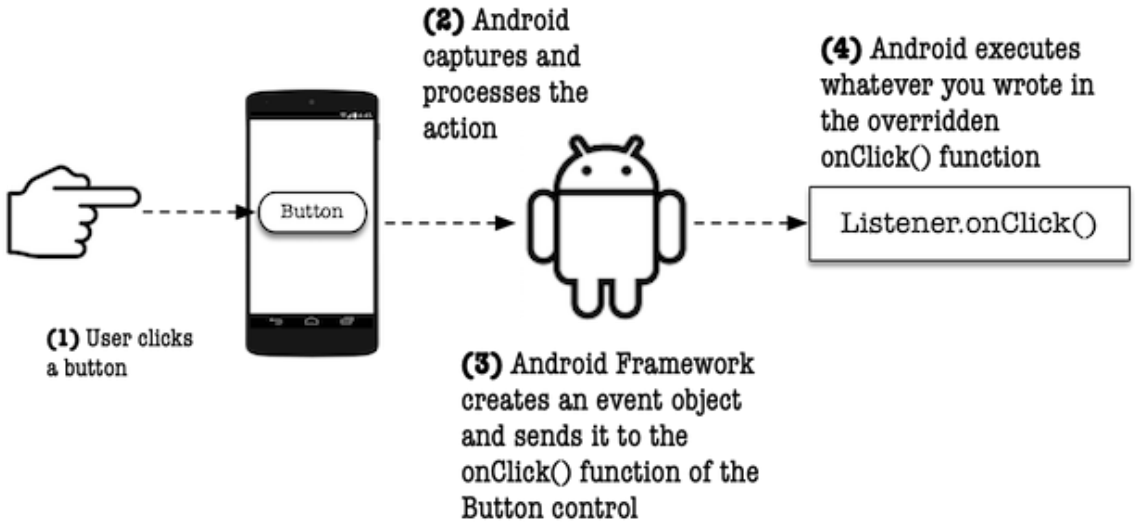
- Sử dụng values để dễ dàng thay đổi và tái sử dụng.
- Tránh hardcode giá trị trực tiếp trong code.
- Hỗ trợ nhiều ngôn ngữ với res/values-xx/strings.xml.
- Tối ưu hình ảnh để giảm dung lượng ứng dụng.

Khái niệm sự kiện

Sự kiện (Event): Tương tác của người dùng với giao diện.

Các sự kiện phổ biến:

- Nhấn nút (Click)
- Nhập dữ liệu (Text Input)
- Chọn giá trị (Selection Change)
- Kéo, thả (Drag & Drop)



Cách lắng nghe sự kiện

- Sử dụng bộ lắng nghe sự kiện như setOnClickListener, setOnTouchListener.

View object. The user will interact with this

Registration. Calling the setOnClickListener on the Button object tells the Android Framework which listener object to use when something interesting happens

```
btn.setOnClickListener(new View.OnClickListener(){
  @Override
  public void onClick(View view) {
    System.out.println("Hello");
  }
});
```

Listener Object. This is an anonymous object, constructed using an anonymous class. When the user clicks on the button control, it will look for an appropriate handler inside this object.

- Lambda giúp đơn giản hóa cú pháp:

```
button.setOnClickListener { view ->
    println("Button clicked at: ${view.id}")
}
```

Các loại sự kiện trong Android

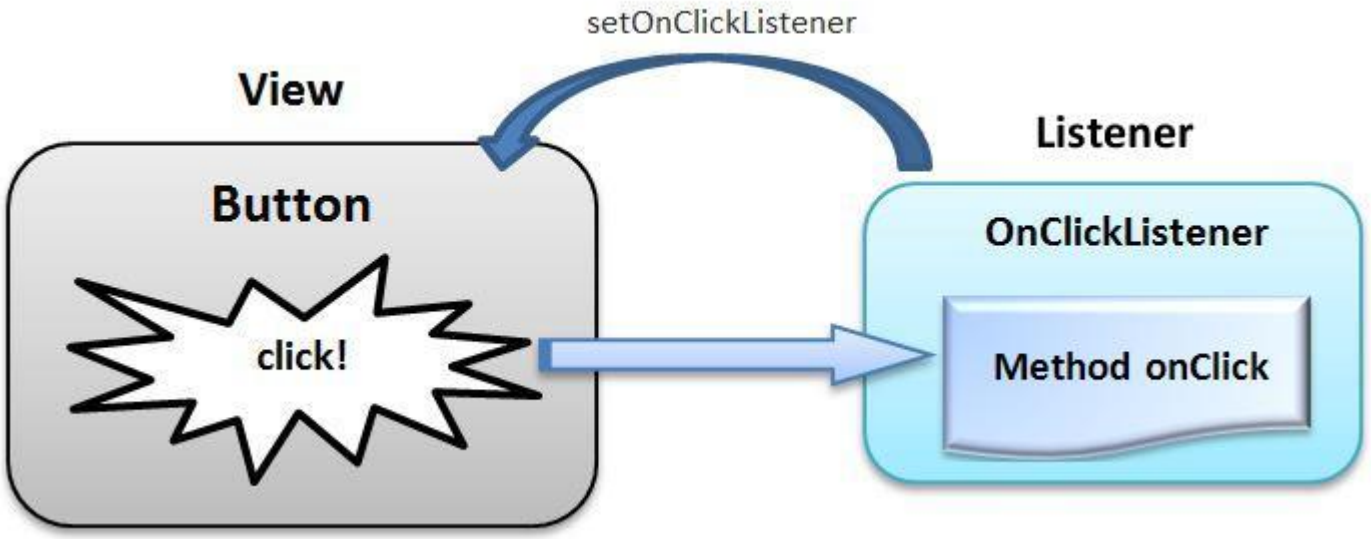
Giao diện (Interface)	Phương thức (Method)	Mô tả
View.OnClickListener	onClick()	Được gọi khi người dùng nhấn hoặc giữ một phần tử giao diện
View.OnLongClickListener	onLongClick()	Giống như click nhưng thời gian nhấn lâu hơn
View.OnFocusChangeListener	onFocusChange()	Gọi khi người dùng chuyển đến hoặc rời khỏi một phần tử
View.OnTouchListener	onTouch()	Nhận biết cử chỉ vuốt lên/xuống hoặc chạm vào màn hình
View.OnCreateContextMenuListener	onCreateContextMenu()	Được gọi khi tạo menu ngữ cảnh (context menu) sau một lần nhấn giữ lâu

Xử lý sự kiện với Button

- Button thực hiện lệnh khi nhấn.

- Ví dụ:

```
button.setOnClickListener {
    Snackbar.make(it, "Button clicked!", Snackbar.LENGTH_SHORT).show()
}
```



Xử lý sự kiện với EditText

- EditText nhận dữ liệu nhập từ người dùng.

- Có thể giới hạn ký tự hoặc kiểm tra đầu vào:

```
edtInput.addTextChangedListener(object : TextWatcher {
    override fun afterTextChanged(s: Editable?) {
        txtResult.text = s.toString()
    }
})
```

Xử lý sự kiện với CheckBox

- CheckBox dùng để chọn nhiều mục.
- Kiểm tra trạng thái với isChecked.

```
checkBox.setOnCheckedChangeListener { _, isChecked ->
    txtResult.text = if (isChecked) "Đã chọn" else "Bỏ chọn"
}
```

Xử lý sự kiện với RadioButton

- RadioButton được nhóm với RadioGroup để chọn một mục duy nhất.

- Ví dụ:

```
radioGroup.setOnCheckedChangeListener { _, checkedId ->
    when (checkedId) {
        R.id.radio1 -> txtResult.text = "Chọn phương án 1"
        R.id.radio2 -> txtResult.text = "Chọn phương án 2"
    }
}
```

Xử lý sự kiện với Spinner

- Spinner giúp chọn một giá trị từ danh sách.

- Xử lý sự kiện khi người dùng chọn mục trong Spinner:

```
spinnerExample.onItemSelectedListener = object : AdapterView.OnItemSelectedListener {
    override fun onItemSelected(parent: AdapterView<*>,
                                view: View?,
                                position: Int,
                                id: Long) {
        txtResult.text = "Bạn chọn: ${parent.getItemAtPosition(position)}"
    }
    override fun onNothingSelected(parent: AdapterView<*>) {}
}
```

Quản lý nhiều sự kiện đồng thời

- Có thể lắng nghe nhiều sự kiện trên một View.

- Kết hợp sự kiện click và cảm ứng.

Ví dụ:

```
button.setOnTouchListener { v, event ->
    println("Touch event detected")
    false
}

button.setOnClickListener {
    println("Button clicked")
}
```

Ví dụ khi xử lý sự kiện

Tạo giao diện cho thành phần Username:

- android:hint: Hiển thị gợi ý nhập liệu
- android:id: Định danh để xử lý trong code

```
<EditText
    android:id="@+id/edtUsername"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Nhập tên đăng nhập" />
```

Ví dụ khi xử lý sự kiện

Tạo EditText cho password:

android:inputType="textPassword": Để nhập mật khẩu

Mật khẩu sẽ được che lại khi nhập

```
<EditText
    android:id="@+id/edtPassword"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Nhập mật khẩu"
    android:inputType="textPassword" />
```

Ví dụ khi xử lý sự kiện

Tạo Button Đăng nhập:

Khi nhấn vào **Button**, sẽ kiểm tra thông tin đăng nhập

```
<Button
    android:id="@+id/btnLogin"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Đăng nhập" />
```

Ví dụ khi xử lý sự kiện

Tạo TextView hiển thị lỗi:

android:visibility="gone": Ẩn TextView ban đầu

Sẽ hiển thị khi có lỗi

```
<TextView
    android:id="@+id/txtError"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textColor="@android:color/holo_red_dark"
    android:visibility="gone" />
```

Ví dụ khi xử lý sự kiện

Lắng nghe sự kiện nút đăng nhập

- setOnClickListener: Xử lý sự kiện khi nhấn vào Button
- trim(): Xóa khoảng trắng đầu và cuối chuỗi
- Hiển thị lỗi nếu username hoặc password trống

```
btnLogin.setOnClickListener {
    val username = edtUsername.text.toString().trim()
    val password = edtPassword.text.toString().trim()

    if (username.isEmpty() || password.isEmpty()) {
        txtError.text = "Tên đăng nhập và mật khẩu không được để trống"
        txtError.visibility = View.VISIBLE
    } else {
        txtError.visibility = View.GONE
    }
}
```

Ví dụ khi xử lý sự kiện

Kiểm tra mật khẩu có ít nhất 6 ký tự

- Nếu mật khẩu dưới 6 ký tự, hiển thị lỗi
- Nếu hợp lệ, ẩn lỗi và tiếp tục xử lý đăng nhập

```
if (password.length < 6) {
    txtError.text = "Mật khẩu phải có ít nhất 6 ký tự"
    txtError.visibility = View.VISIBLE
} else {
    txtError.visibility = View.GONE
}
```

Ví dụ khi xử lý sự kiện

Hiển thị thông báo đăng nhập thành công

Toast.makeText(): Hiển thị thông báo ngắn trên màn hình

```
if (username.isNotEmpty() && password.length >= 6) {
    Toast.makeText(this, "Đăng nhập thành công!", Toast.LENGTH_SHORT).show()
}
```

Ví dụ khi xử lý sự kiện

Thành phần giao diện của ứng dụng Login

Ứng dụng đăng nhập gồm:

- **EditText**: Nhập **username** và **password**
- **Button**: Nút **Đăng nhập**
- **TextView**: Hiển thị thông báo lỗi

Ví dụ khi xử lý sự kiện

Xử lý sự kiện khi nhập password

- TextWatcher: Theo dõi sự thay đổi của EditText
- Kiểm tra password ngay khi nhập, không cần nhấn nút

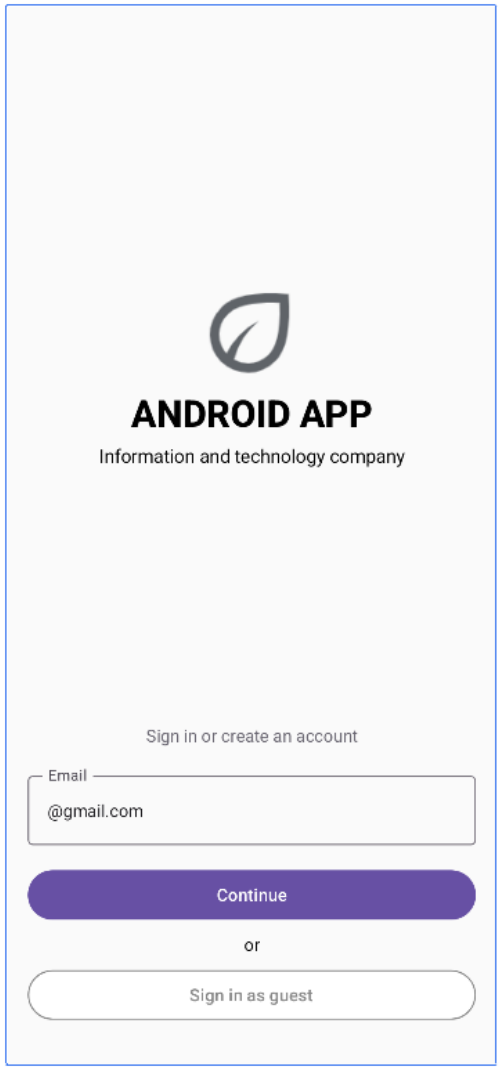
```
edtPassword.addTextChangedListener(object : TextWatcher {
    override fun afterTextChanged(s: Editable?) {
        if (s?.length ?: 0 < 6) {
            txtError.text = "Mật khẩu phải có ít nhất 6 ký tự"
            txtError.visibility = View.VISIBLE
        } else {
            txtError.visibility = View.GONE
        }
    }
})
```

Ví dụ khi xử lý sự kiện

Chạy thử: Trường hợp nhập thiếu thông tin

- **Lỗi**: "Tên đăng nhập và mật khẩu không được để trống“

Sinh viên viết -> chạy phần mềm để thấy kết quả

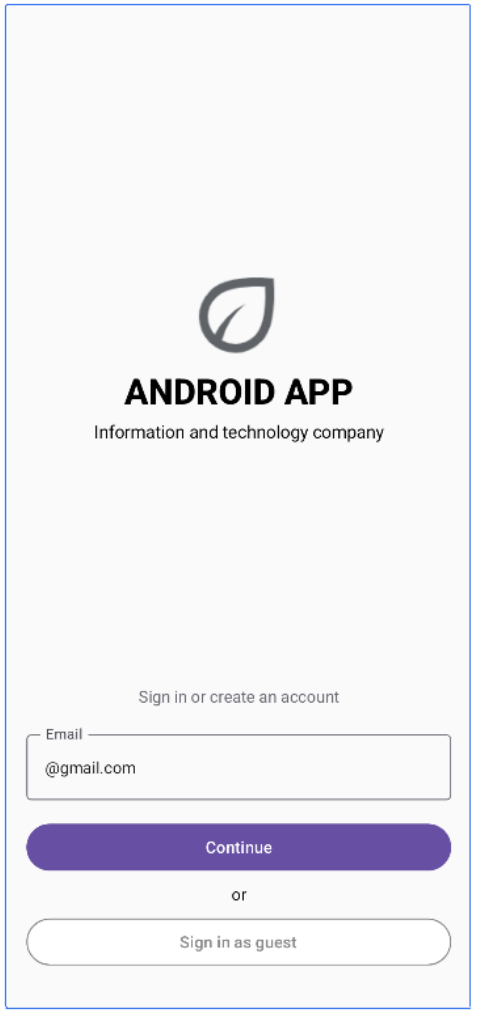


Ví dụ khi xử lý sự kiện

Chạy thử: Trường hợp nhập mật khẩu quá ngắn

- **Lỗi**: "Mật khẩu phải có ít nhất 6 ký tự“

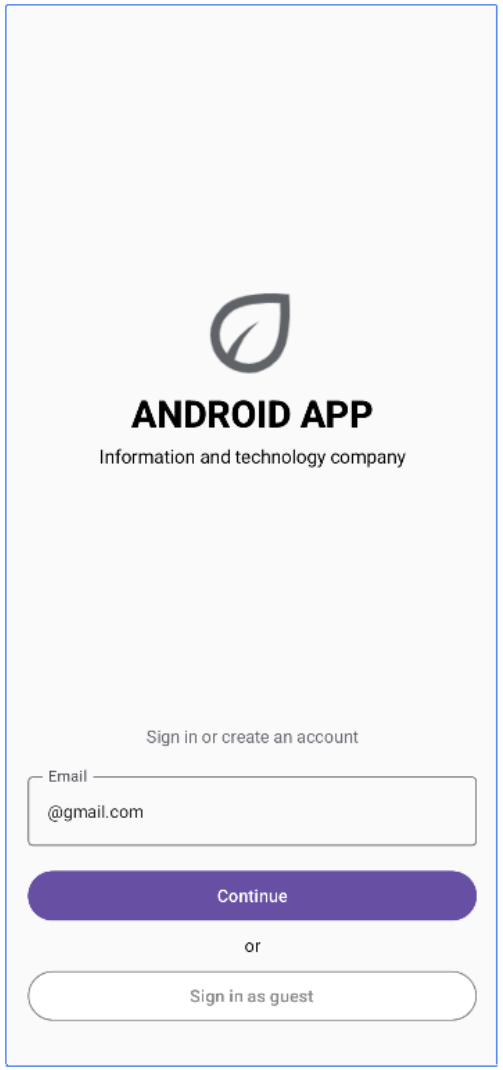
Sinh viên viết -> chạy phần mềm để thấy kết quả



Ví dụ khi xử lý sự kiện

Trường hợp nhập đúng thông tin

- **Hiển thị**: "Đăng nhập thành công!“
- *Sinh viên viết -> chạy phần mềm để thấy kết quả*



Checkbox và RadioGroup để thu thập lựa chọn

- Checkbox và RadioGroup giúp người dùng chọn mục.

Ví dụ:

```
radioGroup.setOnCheckedChangeListener { group, checkedId ->
    println("Radio button selected: $checkedId")
}
```

Hướng dẫn debug sự kiện trong Android Studio

- Sử dụng Logcat để theo dõi sự kiện.

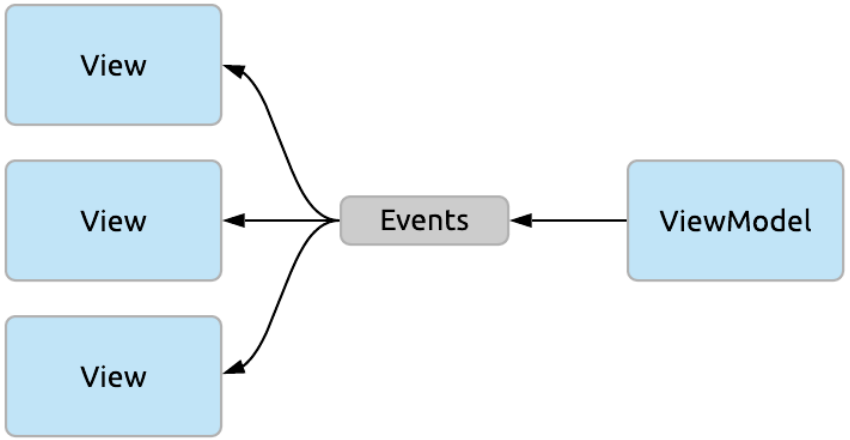
- Sử dụng Debugger để kiểm tra trạng thái sự kiện.

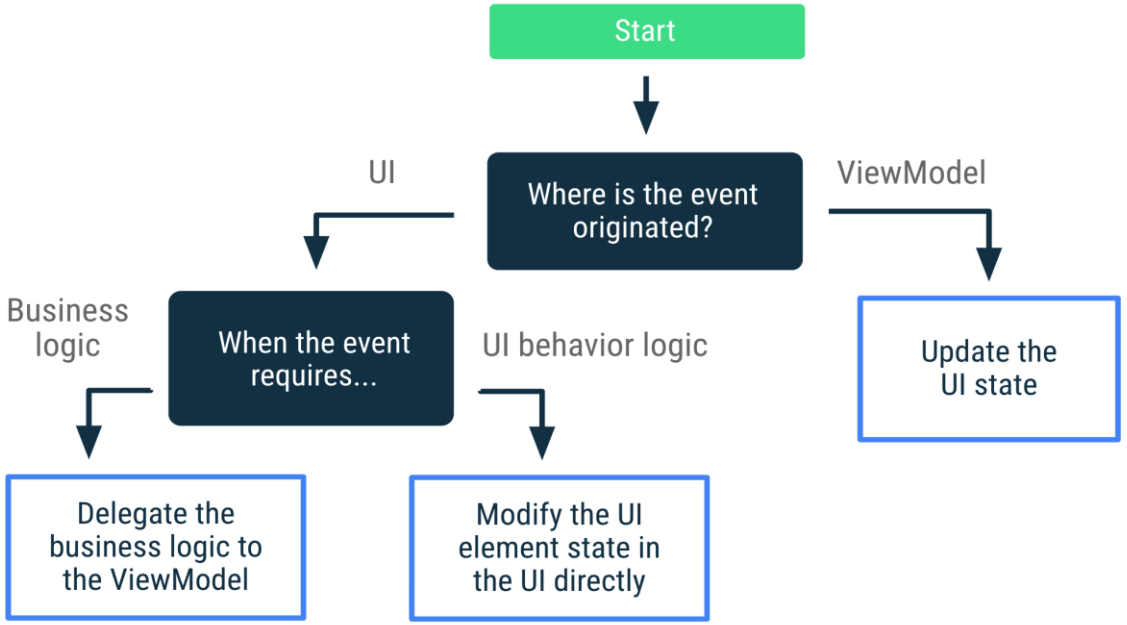
Ví dụ:

```
Log.d("EventDebug", "Button clicked")
```

Các best practices trong xử lý sự kiện

- Tách logic xử lý sự kiện ra khỏi UI.
- Sử dụng ViewModel để quản lý trạng thái sự kiện.
- Tránh xử lý nặng trên UI Thread.





Câu hỏi ôn tập

Các câu hỏi tổng hợp:

1. Hãy nêu sự khác biệt giữa onClickListener và LiveData.
2. Làm thế nào để truyền dữ liệu giữa hai Activity?
3. Hãy viết một ví dụ sử dụng Navigation Component để chuyển Fragment.

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

Thank you!

THÁI NGUYÊN, THÁNG 10 NĂM 2024